PROMPTFORGE STUDIO

Official Capstone Dossier

End-to-End Generative AI Application with Advanced Prompt Engineering

Project	PromptForge Studio - prompt engineering workbench
Author	Ganesh S
Date	October 2026
Contents	Problem statement, architectural implementation, prompt strategies, benchmark evaluations, exportable artifacts

1. Problem Statement

Teams building generative AI features struggle to move from ad-hoc prompt tweaking to a disciplined, production-style workflow. Prompts live in scattered notes, there is no memory of what was tried, no quality measurement, and no safe behaviour when the model is asked something out of scope. PromptForge Studio solves this by packaging the full prompt engineering lifecycle into one interactive application.

Goals

- Define a business use case and design a system + user prompt architecture for it.
- Manage multi-turn context and conversation memory.
- Version prompts and support experimentation (save, reload, compare).
- Evaluate every AI response with automatic quality metrics.
- Handle errors and off-topic queries with explicit fallback prompts.
- Ground answers in an external data source (editable knowledge base).
- Run as a deployment-ready application with full documentation.

2. Architectural Implementation

The app is a full-stack React application (TanStack Start) with a managed cloud backend (Lovable Cloud) for persistence. All AI calls run server-side through the Lovable AI Gateway, so the API key never reaches the browser.

Request flow

```
Browser chat UI
-> sendMessage() server function (TanStack Start)
-> builds prompt: system + tone + fallback + knowledge base + history
-> Lovable AI Gateway (Responses API, streamed server-side)
-> answer returned, then scored by a second AI evaluator call
-> conversation, messages and scores persisted to the database
-> UI renders answer + FALLBACK / QUALITY / SOURCES badges
```

Components

Layer	Responsibility
Chat UI (src/routes/index.tsx)	Preset selection, prompt editing, live chat, quality metrics panel
Server functions (studio.functions.ts)	sendMessage, listPromptVersions, savePromptVersion, loadConversation
AI gateway (gateway.server.ts)	Streamed Responses-API calls through the Lovable AI Gateway
Database (Lovable Cloud)	prompt_versions, conversations, messages tables with row-level security
Docs page (src/routes/docs.tsx)	In-app documentation of architecture, prompt strategy and usage

3. Prompt Strategies

Every conversation is driven by a five-part prompt architecture. The user can edit each part live and save the combination as a version.

Element	Strategy
Use case	Frames the business context so the model adopts the right persona and scope.
System prompt	Defines role, rules, safety constraints (e.g. never ask for PINs/OTPs) and output format.
Fallback prompt	Explicit instruction for out-of-scope queries - a graceful, on-brand refusal instead of hallucination.
Tone	A short style directive (e.g. 'friendly and reassuring') injected into every request.
Knowledge base	External data source: one fact per line, injected as grounding context; answers cite SOURCES.
Memory	Full conversation history is sent each turn, enabling follow-ups and context retention.

Built-in prompt architectures (presets)

Six ready-made architectures ship with the app: Customer Support (NovaBank), HR Policy Assistant, Course Advisor, E-commerce Assistant (ShopKart), Travel Planner (WanderWise) and Fitness Coach (FitTrack). Each demonstrates a different persona, constraint set, tone and grounding knowledge base.

Evaluation strategy

A second AI call acts as an evaluator: it receives the system prompt, the user message and the answer, and returns strict JSON with an overall score (/100), relevance, clarity and tone-match (/10 each) plus one-line feedback. Scores are shown per message and averaged in the metrics panel.

4. Benchmark Evaluations

The application was verified end-to-end in a real browser session against the live AI gateway and database.

Test	Input	Expected	Result
Grounded answer	"How do I block my debit card?"	Numbered in-app steps, no PIN/OTP request	**PASS - quality 94/100**
Context memory	Follow-up: "how long until it arrives?"	Uses history to know 'it' = replacement card	**PASS - answered 5-7 business days from knowledge base**
Fallback	"What is the weather today?" (off-topic)	Fallback prompt triggers, polite refusal	**PASS - FALLBACK badge shown**
Versioning	Save and reload a prompt version	Version persisted and restores config	**PASS**
Logging	Any conversation	Turns stored with prompt version reference	**PASS**

5. Requirements Coverage

#	Required feature	Where delivered
1	Business use case definition	Use-case field + 6 presets
2	System and user prompt architecture	Editable system prompt, tone, fallback; user turns
3	Context management and memory	Full history sent each turn; conversation persistence
4	Prompt versioning and experimentation	Save/reload versions, preset switching
5	Response evaluation and quality metrics	AI evaluator scores + metrics dashboard
6	Error handling and fallback prompts	Fallback instruction + FALLBACK badge + error toasts
7	External tools / data sources	Knowledge-base grounding with SOURCES badge
8	Deployment-ready workflow	Full-stack app on Lovable Cloud, one-click publish
9	Complete project documentation	In-app Docs page + this dossier

6. Exportable Artifacts

The project ships with the following reusable artifacts:

- Six prompt architecture presets (system prompt + fallback + tone + knowledge base) that can be copied into any LLM application.
- A reusable AI gateway module (gateway.server.ts) for streamed, server-side Responses-API calls.
- A database schema for prompt versions, conversations and messages with row-level security.
- An evaluation prompt template that returns strict JSON quality scores.
- In-app documentation (Docs page) and this capstone dossier PDF.

Skills demonstrated

Prompt architecture design, context and memory management, grounding with external data, LLM-as-judge evaluation, fallback engineering, prompt versioning, and full-stack deployment - directly relevant to Prompt Engineer, Generative AI Engineer and AI Product Developer roles.